Weekly report — fixed-income pricing module

Date: 2026-08-30 · **Covers:** the six rows you marked on the `Pivot of Corp Bonds` sheet

You went down the coupon-type pivot with us and marked six rows **no** — the ones the restructured code did not yet cover. All six are now covered, and this report answers them cell by cell.

One of the six turned out not to be a data problem at all. It was a bug in how we read a market-data file, and it was also quietly costing us a bond in the row you had already marked **finished**. That is section 3, and it is the most important thing here.

As before: everything except **section 6** is written to be read without the code. Section 6 is for the engineering team and the finance reader can skip it.

---

1. Your six cells, answered

A word on the counts first, because several different denominators are easy to confuse. The pivot counts **rows on the Corporate Bonds sheet** (676 of them). That sheet lists some securities more than once — 60 of them — so 676 rows are 616 distinct bonds. Each row below therefore shows the pivot count, the number of distinct securities behind it, how many are live holdings, and how many now produce a full set of numbers.

Cell	Coupon type	Pivot rows	Securities	Held	Priced	Not priced
F12	Fixed → Floating	5	5	5	4	1
F13	7.00% before 01-Mar-2006, 7.50% after	2	**1**	1	1	—
F14	GBP LIBOR + Spread	1	1	1	**1**	—
F15	Reference Rate + Spread	12	12	12	11	1
F16	EURIBOR + Spread	9	9	9	5	4
F20	Step-up schedule	1	1	1	1	—
	total	**30**	**29**	**29**	**23**	**6**

A correction to how we put this to you. An earlier draft of this table showed F13 as "2 rows, 1 held", which reads as though one of those two is not a holding. It is not: your sheet lists the *same* bond twice. One bond, held, priced. Every one of the 29 distinct securities behind your six cells is a live holding — none of them is missing from the book.

The six that do not price are not a modelling gap. Five are bonds that pay a fixed coupon for some years and then switch to a floating one, and the *margin they pay after the switch* is not in the workbook and is not public — it is on the Bloomberg list already with you. Each is carried at the custodian's price with that reason attached, and each becomes a priced bond the moment the margin arrives: it is one cell in a data file, with no code change at all. The sixth is a defaulted bond, which is carried at its recovery mark

on purpose — a spread calculated against a borrower who has stopped paying would be a number without a meaning.

We also, for free, covered the six rows immediately above yours (`Fixed → Reset` , rows 6–11), because they are the same instrument shape and share the same engine. Three of those six now price; the other three are waiting on the same missing margins.

2. What these coupon types actually are

Four families sit behind your six cells, and they need genuinely different treatment:

- **Floating-rate note** (F14, F15, F16) — the coupon is not fixed at all. It resets periodically to a market interest rate plus a contractual margin. This has a consequence worth knowing: such a bond keeps re-setting its own interest-rate risk away, so a thirty-year floating note behaves, for rate purposes, like a bond maturing at its next reset. Our numbers show exactly that, and the module reports the next reset date beside the risk figure so the two can be checked against each other.
- **Fixed → Floating** (F12) — fixed for some years, then floating. Priced as one bond with one credit spread across both halves, because it is one borrower's one promise. Splitting the spread in two would be inventing a second borrower.
- **Stepped / step-up** (F13, F20) — the coupon changes over time on a schedule that is **known in advance**. There is no option and no uncertainty here, so these are not a new model at all: they are ordinary bonds priced with a table of coupons instead of one coupon. Saying so is worth more than building something.
- **Zero / structured** (F21, which you had already annotated "just corp bond (fixed)") — agreed, and that is how it is treated.

3. F14 was our bug, not missing data — please drop that request

We have been telling you the single British-pound bond could not be priced because our UK interest-rate curve "was not arbitrage-free". That was wrong, and we should correct it plainly.

The market-data files we were given are not all written the same way. Most store interest rates as decimals — `0.0304` meaning 3.04%. The **British-pound file stores them as percentages** — `3.04` meaning 3.04%. Our loader assumed decimals for every file, so it multiplied the pound rates by a hundred and produced a curve with three-year rates near 200%. The bootstrapping step then, correctly, refused to build a curve out of it, and reported the only thing it could see: that the curve was not arbitrage-free. We read that as a statement about the data, and raised a request for a replacement UK curve.

The curve was always fine. Read correctly, the 31 March 2009 file gives 1.18% at two years, 2.34% at five, 3.16% at ten and 4.16% at thirty — the actual gilt market that day.

What this changes:

	Before	Now
France Télécom 7.50% of 2011 (GBP) — your F14	not priced	spread **205.3 bp**, rate sensitivity 1.86 years
A UK 5.50% bond of 2033 (GBP)	**missing from the output entirely**	spread **197.3 bp**, rate sensitivity 12.55 years
Corporate bonds in the output	564	**565**
Of those, fully priced	553	**555**

The second bond is the part that matters beyond this week. It was not flagged, it was silently **skipped** — and it is a plain fixed-coupon bond, which is to say it belongs to the row you had already marked *finished*. A completeness count can be wrong in a direction that no report shows you, and this one was.

Two independent checks that the new numbers are right. The bond's spread measured against the *dollar* curve is 279.9 bp and against its own *sterling* curve 197.3 bp; the 83 bp gap is precisely the difference between gilt and Treasury yields at that maturity. And the France Télécom spread of 205 bp sits exactly where a single-A rated telecom belonged in March 2009, next to its own euro-denominated bonds.

We also checked every one of the 26 market-data files this way, at three dates each. One other file — Danish krone — has the same percentage convention. It is not used by this portfolio, but it is now declared, so the next person to reach for it does not repeat this. And the loader now **refuses** any file whose rates come out above 100%, naming the units as the cause, so this class of mistake cannot again disguise itself as a statement about the market.

Action for you: the request for a replacement UK curve can be withdrawn. Nothing else on the outstanding Bloomberg list changes.

3.1 A second one of the same kind — caught before it reached you

Having found one bond that had gone missing without a trace, we went looking for others of the same shape. There was one, and this time we found it **before** it affected any number we have shown you.

Five bonds in the portfolio are callable — the borrower may repay early. Three are priced on the option model, and one is waiting for its call terms, which are on the outstanding data request. That accounts for four. **The fifth had fallen between two rules.** One part of the code treated a call less than a week before maturity as economically irrelevant and priced the bond normally; another part only sent bonds to the option model when the call was more than a year before maturity. A bond whose call sits 90 days before maturity satisfied neither, so it was priced by nothing — and, unlike a flagged bond, it produced no message anywhere. It is a real holding: 850,000 nominal, marked at 85.12.

It had even been written down, once, in our own work log, as a "minor loose end" — and then it fell out of every count that followed. That is the lesson rather than the bond: a completeness figure can be wrong in a direction no report shows you, and the same failure had now happened twice.

Two things changed, neither of which moves a price:

- **the counting is now mechanical.** Every bond in the portfolio must leave the calculation either with a number or with a named reason. That is checked as a set — not as a total, because a total of "3 priced + 2 skipped = 5" balances perfectly even with the wrong bond in the wrong place, which is exactly

how this one stayed hidden. The check runs on every production run and stops it if any bond is unaccounted for.

- **the two rules became one.** Deciding *whether* a bond is callable now happens in one place only; the option model simply prices everything it is sent.

And a third thing, which is the part worth knowing about the model itself. When we sent that bond to the option model to see what it was worth, it returned a value for the early-repayment right of **exactly zero** — which looks like "the right is worthless". It was not. The option model makes its decisions on the bond's coupon dates, and this bond's call date falls in the final three months, after the last coupon. There was no date on which the model could consider it, so it never did. The number was not an answer; it was the absence of a question.

The model now **refuses** such a bond rather than pricing it as an ordinary one — which is why the fifth callable bond is still not priced, and is now named and explained instead. We checked all eight bonds that carry call terms today: none of them is affected, so no number you have been given is wrong. Deciding what that bond should ultimately be worth needs either a change to the option model's calendar or a documented rule about very short call windows; both are real decisions and we have not taken either quietly.

4. One change to the Excel connection

Last week the spreadsheet could ask for one kind of bond. It can now ask for any of seven — plain, stepped, floating, fixed-then-floating, callable, puttable and sinking-fund — by naming the type in the request. Everything else is identical: same single entry point, same request and answer format, same error reporting.

We made this change **once**, covering all seven, rather than twice. And a request that does not name a type is still treated as a plain bond, so **nothing that works today stops working** — including the spreadsheet as it stands, whose 23 automated Excel checks all still pass untouched.

Each type reports the one or two extra numbers only it has: the next reset date for a floating note, the switch date for a fixed-then-floating one, and — for the three types where somebody can repay or return the bond early — the volatility answer from your last round, in both directions.

There is one thing the module will now refuse rather than answer. If a fixed-then-floating bond is sent without its post-switch margin, we do not price it with a zero. A guessed margin produces a confident-looking number for a bond that is half-modelled, and nothing in the output would say so.

5. What we deliberately did not do

- **The demonstration spreadsheet still asks for plain bonds only.** The engine and the message format handle all seven types, but putting seven bond types on a worksheet is a layout question, and we would rather do it once with you than guess. The interface document lists exactly which fields each type adds.
- **No bond was re-classified to make a count look better.** The six unpriced bonds stay unpriced and named.

- **We did not touch the holdings workbook.** Your column F is your marking; section 1 is the evidence for updating it, and the update is yours to make.

6. For the engineering team

Structure. Three engines moved into the template layout this week — `core/pricing/floating.py`, `core/pricing/hybrid.py` and `core/pricing/coupon_schedule.py` — each a verbatim move with the old import path left as a compatibility shim. Three thin per-product wrappers (`assets/corporate/{floating,hybrid,stepped}.py`) provide the one-simple-function-per-output surface, with numbered input blocks in every docstring.

The move also closed a layering violation: `core/pricing/cashflows.py` had been importing from the legacy `pricing` package, i.e. the core reaching upward into a not-yet-migrated layer. It now imports a sibling.

Interface. `analyze_payload(request) → response`, dispatching on `bond.instrument_type`. Adding an engine later is one map entry and one function — no new endpoint, no second envelope, no second error vocabulary. `analyze_vanilla_payload` is retained as an alias.

Verification.

Automated checks	**300**, in about 24 seconds (223 at the start of the week)
Population accounting	enforced at run time — every bond leaves with a number or a named reason, checked as a SET of identifiers, not as a total
Production outputs after each migration step	byte-identical to a pre-change baseline, all five driver files
Endpoint vs direct function call	asserted equal with `==`, per instrument type, not a tolerance
Compatibility shims	asserted to re-export the *same object*, not an equivalent one

Two details worth knowing before you run it:

- **Cross-platform determinism has a limit worth writing down.** Full 565-bond driver outputs from Windows and from the Linux server differ by up to **3.6e-8 relative**, and entirely in the convexity column — a second difference divided by the square of a one-basis-point bump amplifies a last-bit rounding difference by 10^8. Prices, spreads and durations agree to about 1e-12, and every text column is identical. A byte-for-byte comparison across platforms will therefore show a difference that is not a regression; compare on one platform, or compare with a tolerance.
- **The floating-rate duration has two exact regimes**, and the sign flips between them. Supply the already-fixed current coupon and the answer is *plus* the time to the next reset; omit it and the coupon is projected off the curve, the bump reprices it too, and the answer is *minus* the time since the last reset. Both are within one coupon period and both are correct; only the interpretation differs. It is pinned by tests.

```
python -m pytest -q                                              300 checks, ~24 s
python scripts/price_json.py --input request.json --output response.json
powershell -File integrations/excel_vba/tests/Run-BridgeTests.ps1  23 Excel checks
```

7. Next

- **Mortgage-backed securities** remain the largest outstanding block, and they are waiting on the pool data pull, not on us — the engine skeleton is built to the exact field list.
- **The demonstration spreadsheet for the new bond types**, once you tell us how you would like them laid out.

8. One question for you

Do you want the extra bond types on the demonstration sheet, and if so, in what shape? One sheet with a bond-type dropdown that shows and hides the fields each type needs, or a separate small sheet per type? We have no preference and it is a couple of hours either way — but it is your team's daily view, so it should be your call.